MP2 Report — Selecting Reliable LLM Reasoning

DASE7506 · Advanced Machine Learning Methods · Fall 2026

- **Student ID:** `3036706612`
- **Final method:** permutation-equivariant self-attention over whole-trace embeddings, plus two hand-crafted reasoning-consistency features, trained with dropout.
- **Validation accuracy:** 92.98% · **Test accuracy:** 79.01%

1. Problem and the question I chose

A language model produces four solutions to the same maths problem; they all look plausible, each shows several calculations, and they still disagree on the answer. My program must pick one candidate index (0–3) per question, and the official scorer checks only that candidate's **final numeric answer** — it does not judge whether the reasoning is valid.

The guide poses several questions worth investigating. I chose to organise my whole project around one of them:

> ***"Does reading the reasoning actually help?"***

with two follow-ups that emerged naturally as I worked:

1. **"Should a solution be judged on its own?"** — does comparing the four candidates help more than scoring each in isolation?
2. **"What makes a wrong answer look convincing?"** — is it arithmetic slips, or coherent-but-wrong reasoning?

My working hypothesis was: *the reasoning contains the useful signal, and it can only be exploited by comparing candidates rather than scoring them independently.* The rest of this report is the record of testing that hypothesis, step by step, with the evidence for each claim.

2. Setup and evaluation protocol

Data. 4,755 training / 798 validation / 467 test questions, four candidates each, no split overlap. Inputs are the question, four candidate texts, and fixed whole-trace embeddings `arrays['embeddings']` of shape `[N, 4, 1024]` (the candidate's reasoning and answer fused into one vector — I cannot separate them without an encoder, which the constraints forbid).

Task and metric. Exactly one index 0–3 per question. The fixed scorer parses the selected candidate's final answer (`####` or `\boxed{}`, canonicalised via `Fraction`) and compares it to the reference. Any correct candidate earns the point, so the task is really "recognise a good answer among the available ones." Metric is selection accuracy in percent.

Loss. Every learned method uses the same objective. For one question with scores `s ∈ R^4` and a boolean mask `t` of correct candidates:

```
loss = logsumexp(s) − logsumexp(s masked to correct candidates)
```

This is a softmax-margin over candidates: it raises the correct candidates' scores relative to *all* candidates, and it can be positive when more than one candidate is correct. It is identical across all my models, so differences between runs are attributable to the **features** and the **head**, not the loss.

Hyper-parameters (fixed across all trained models): hidden dim 128, Adam lr 0.01, weight decay 1e-4, batch 256, up to 100 epochs, seed 42, early stopping by best validation accuracy. Training is CPU-only.

Constraints I respected throughout: training only on the supplied training inputs/references; validation used solely for development and model selection; no external data or pretrained weights; no LLM calls at inference; the frozen method reads only `bank`/`arrays`.

3. Method evolution

I built the method in six steps. Each step was motivated by a specific question or observation from the previous step, and I record the hypothesis → result → lesson for each. All accuracy numbers are on the **validation** split (798 questions), the only split used for development.

#	Method	Val acc	Wrong
1	Majority vote on parsed final answers (no learning)	86.22%	110
2	Two-layer MLP over embeddings + vote feature	91.73%	66
3	Self-attention over the four candidates	92.11%	63
4	+ binary "has an arithmetic error" feature	92.86%	57
5	+ error *count* + final-answer-consistency feature	**92.98%**	56
6	Dropout (regularisation, used from step 3 onward)	—	—

Step 1 — Majority vote (the floor)

Question: how far can I get without reading the reasoning at all, just by trusting agreement among the final answers?

Design. Parse each candidate's final answer; pick the one that occurs most often (ties → first occurrence). Unparseable answers count as disagreement. No weights, no checkpoint.

Result. 86.22%. **Lesson.** Agreement among final answers is already a strong signal — four sampled answers usually cluster around the right value — but it caps out where the wrong answers happen to agree with each other, and it has no access to *why* an answer was reached.

Step 2 — MLP over the reasoning embeddings

Question (the central one): does reading the reasoning actually help?

Design. Replace the vote with a learned scorer: a two-layer MLP that maps each candidate's L2-normalised whole-trace embedding (1024-d) to a score, and take the argmax. I keep the vote as a one-dimensional extra feature, since it is cheap and informative.

Result. 91.73% — **+4.51 points over the vote baseline.**

Lesson. This is the direct answer to my central question: the reasoning representations account for most of the gain. To be sure the gain comes from *reasoning* and not just from replacing a heuristic with a learned model, I ran two control ablations (Section 4.1): an **answer-only MLP** (same architecture, features built only from the parsed final answers) scored ≈ 86%, essentially *identical to the vote*, while **embeddings without the vote feature** scored 90.35%. So the reasoning signal is real and worth ~4–5 points, and the vote feature adds a further ~1.4 on top of it.

Step 3 — Comparing candidates instead of scoring them in isolation

Question: should a solution be judged on its own?

Design. The MLP scores each candidate independently (`Linear(1025 → 128) → ReLU → Linear(128 → 1)` per candidate, no interaction). I replaced the head with a permutation-equivariant one that lets the four candidates interact: a one-layer self-attention (`TransformerEncoderLayer`) over the four candidate features, no positional encoding, so the scores are a symmetric function of the set of candidates.

Result. 92.11% (no dropout yet) — a further gain over the independent MLP. With dropout fixed at 0.2 (Section 4.3), attention reaches 92.61% vs. the independent MLP's 91.73%, and a DeepSets head (per-candidate encoder + mean-pooled global summary) reaches 92.23%. **Lesson.** Comparing candidates helps, consistently across two different comparison heads. A candidate that looks fine alone can reveal a missed condition or a conflicting interpretation once it is read next to the other three — this is the "should it be judged on its own" question, and my answer is *no*.

Step 4 — Adding an explicit arithmetic-error signal

Question (from "what makes a wrong answer look convincing"): can I exploit *visible* arithmetic mistakes?

Design. I wrote a deterministic checker that normalises the reasoning (LaTeX/markdown/Unicode), extracts every explicit `X = Y` / `X ≈ Y` equation, evaluates the left-hand side with a safe AST evaluator, and flags it when the stated right-hand side disagrees (relative tolerance 1e-3, or 1e-2 for ≈). I add a **binary** feature per candidate: 1 if it contains at least one such error, 0 otherwise.

Result. 92.86%. **Lesson.** A modest gain, but I first measured the ceiling: inspecting the 63 validation errors of the step-3 attention model, **only 5 (7.94%) contain a visible arithmetic error** (e.g. `3+5+7+9+11=36`, `5600+392=6000`). The other ~92% of errors are *not* arithmetic slips — they are coherent, correctly-computed chains that solve the wrong sub-problem. This directly answers "what makes a wrong answer look convincing": it is almost never bad arithmetic, it is plausible-but-wrong *reasoning*. The arithmetic feature therefore has a low ceiling, which matches the small gain.

Step 5 — Error count + final-answer consistency

Question: can I extract a stronger signal from the reasoning's *internal* structure?

Design. Two refinements. (1) Replace the binary error flag with the **count** of arithmetic errors, so the model can distinguish "one slip" from "a chain full of slips." (2) Add a **consistency** feature: 1 if the right-hand side of the *last* equation in the candidate's own chain disagrees with its *own* stated final answer (relative tolerance 1e-3), else 0. A candidate whose chain computes 14 but whose boxed answer says 13 is internally self-contradictory, and that is a strong "likely wrong" signal — orthogonal to both the vote feature and the arithmetic check.

Result. 92.98% — the final model. **Lesson.** These are small, mostly-noise-level gains (Section 4.2), consistent with the low ceiling measured in step 4. They are "no harm, marginal help," and I keep them because the consistency feature is genuinely a *different* failure mode from the others.

Step 6 — Dropout (regularisation)

Question: why did the attention model overfit?

Observation. During step 3 I noticed a textbook signature: training accuracy climbed to ~97% while validation loss *rose* (to 0.59) and validation accuracy stopped improving — the attention head has enough capacity to memorise the training set. This is why I chose the checkpoint by best *validation* accuracy, not best training loss.

Design. Dropout in the attention head (`p = 0.3`). It is a **training regularisation**, not a new "method step": it has been used in steps 4–5. The comparison below shows why.

Dropout	Best val acc	Note
0	92.11%	overfits; val loss rises to 0.59
0.3	92.98%	stable

Lesson. The dropout is essentially free and necessary for the attention head; without it the model memorises and the marginal gains of the later features would be swamped by the train/validation gap.

4. Ablations and what they answer

Each research question is answered by a controlled comparison, all under the same loss, splits and protocol.

4.1 Does reading the reasoning actually help? — Yes, ~4.5 points

Method (same MLP head)	Val acc
Majority vote on final answers	86.22%
Answer-only MLP (learned, no embeddings)	≈ 86%
MLP over embeddings (no vote feature)	90.35%
MLP over embeddings + vote feature	91.73%

The answer-only *learned* model ties the vote baseline — so the improvement is not "learning beats heuristics," it is specifically the **reasoning representations**. The vote feature contributes a further ~1.4 points on top of the embeddings.

4.2 Should a solution be judged on its own? — No

Head (same embeddings + vote features)	Val acc
Independent MLP	91.73%
DeepSets (comparison)	92.23%
Self-attention (comparison)	92.61%

Both comparison heads beat the independent scorer under matched settings, so the gain is structural (comparison), not a quirk of one architecture. I chose self-attention as the final head.

4.3 Do the explicit checks help? — Marginal, and near the noise floor

Method	Val acc
Attention + dropout 0.2 (no checks)	92.61%
+ binary arithmetic-error feature	92.86%
+ error count + consistency feature	92.98%

I want to be honest about this table: the feature gains (+0.12 to +0.25) are **within the seed-to-seed variance** of the attention head (which I observed to swing ~0.5 points), and the dropout setting is not perfectly aligned across rows (0.2 vs 0.3). The intellectually honest conclusion is: *these hand-crafted features do not hurt and give a small, noisy gain, consistent with the low ceiling I measured in step 4.* That ceiling — only 7.94% of errors contain a visible arithmetic slip — is itself the more interesting finding.

4.4 When does more computation pay off?

- **Comparison heads (step 3):** +0.5–0.9 points for a few hundred thousand extra parameters. Worth it.
- **Hand-crafted features (steps 4–5):** +0.1–0.3 points. Diminishing returns; the deterministic checkers cost nothing at inference but add little.
- **Dropout (step 6):** free, and necessary. The cheapest and clearest win.
- **Ensembling / multi-seed voting:** I evaluated this idea but chose **not** to use it in the final method. It is a standard variance-reduction trick that would push the *score* up without advancing the *research question*, and it risks masking the single-model comparison that this report is about. (Its rationale — averaging cancels each model's independent errors but cannot cancel the common bias — is exactly why it is not a "method" contribution.)

5. Error analysis: what makes a wrong answer look convincing

I instrumented my arithmetic checker over the wrong selections of the step-3 attention model and found the single most informative number of the project:

> ***Only 5 of 63 wrong selections (7.94%) contain a visible arithmetic error.***

The rest are wrong for subtler reasons — a missed condition, a wrong setup, or solving a slightly different problem than the one asked — while their arithmetic is *internally correct*. This is precisely the guide's example: Candidate A multiplies `60 × 3 = 180` correctly but treats the 30-minute stop as travel time; Candidate B accounts for the stop and gets 150. Both multiplications are correct; the difference is *reasoning*, not arithmetic.

This explains the whole trajectory: methods that read the reasoning (embeddings) gain ~4.5 points because that is where the truth lives, while methods that check arithmetic gain almost nothing because arithmetic is rarely where the error lives. It also explains why the **consistency** feature is my preferred check: it catches a candidate whose *conclusion disagrees with its own chain*, which is closer to "wrong setup" than to "wrong arithmetic."

6. Resources and cost

- **Final checkpoint:** `runs/set_feat_attention_dropout03/model.pt`, **1,325,383 bytes ≈ 1.26 MiB** — well under the 16 MiB limit. The model is ~0.33 M parameters (proj 1027→128, one `TransformerEncoderLayer`, head 128→1), stored as float32.

- **Inference:** a single forward pass over the bank (the two hand-crafted features are computed by the same deterministic checkers used in training, with no learned weights). Measured externally on the 467-question test split with four threads (Dell Inspiron 16 7610): **~1.5 s** total (prediction 1.4 s + scoring 0.1 s; repeat runs about 1.5 s) — well under the 30 s budget.
- **Training:** ~50.9 s on a CPU-only laptop (Dell Inspiron 16 7610), 100 epochs, seed 42, torch 2.7.1, numpy 2.5.3, four threads.
- **Reference machine:** the 30 s / 16 MiB limits are measured on a Xeon Platinum 8457C; my assets and per-question cost are dominated by the 1.26 MiB checkpoint and one small forward pass, so the limits are not binding.

7. Reproduction

All commands run from `code/`. Dependencies: `torch`, `numpy` (pinned in `requirements.txt`), Python 3.12.

```
# 1. Majority vote (training-free)
python show.py --method baseline_vote.py --split validation --output-dir runs/show_vote
# 2. MLP over embeddings + vote
python train2.py --output-dir runs/mlp_vote
python show.py --method my_model.py --checkpoint runs/mlp_vote/model.pt --split validation --output-dir
runs/show_mlp_vote
# 3. Self-attention
python train3.py --model attention --output-dir runs/set_attention
# 4. Attention + binary arithmetic-error feature
python train4.py --model attention --calc-binary --no-consistency --dropout 0.3 --output-dir
runs/set_calc_attention_drop03
# 5. Attention + error count + consistency  (FINAL)
python train4.py --model attention --dropout 0.3 --output-dir runs/set_feat_attention_dropout03
# Final test prediction (frozen method)
OMP_NUM_THREADS=4 OPENBLAS_NUM_THREADS=4 MKL_NUM_THREADS=4 \
python predict.py --method my_model4.py --checkpoint runs/set_feat_attention_dropout03/model.pt \
  --split test --output runs/test-predictions.json
```

The frozen method is `my_model4.py` (exposes `predict(bank, arrays, checkpoint)`); the training script is `train4.py`; the comparison heads and the deterministic arithmetic/consistency checkers live in the same file. The submitted checkpoint matches the step-5 run above (seed 42, dropout 0.3, error count + consistency features, vote feature on).

8. AI and reuse disclosure

- **AI assistance.** I used Claude Code (Anthropic) throughout as a coding and analysis assistant. I reviewed and understand every component, ran all experiments myself, and verified every number in the tables against the terminal logs.
- **Reuse.** `common.py`, `predict.py`, `score_predictions.py`, `check_data.py` and the data are supplied by the course and left unchanged. The selection loss, the permutation-equivariant heads, and the arithmetic checker are my own implementation; no external model weights or datasets are used beyond the supplied inputs, and the embeddings are the supplied fixed whole-trace representations.
- **Data.** Questions/references derive from GSM8K (MIT); candidate solutions and embeddings were prepared with Qwen3-0.6B by the course. Kept under the supplied `data/LICENSE.txt` / `data/NOTICE.txt`.

9. Conclusion

Starting from "does reading the reasoning actually help?", I found a clean, evidence-backed answer to each question I asked:

3. **Yes — reading the reasoning is worth ~4.5 points** (86.22% vote → 91.73% embeddings), and it is specifically the reasoning representations, not the learned model, that delivers it.
4. **No — a solution should not be judged on its own** (91.73% independent → 92.61% comparison).
5. **Wrong answers are convincing because their reasoning is coherent, not because their arithmetic is wrong** (7.94% of errors contain a visible arithmetic slip), which bounds how far arithmetic/consistency checks can go.
6. **More computation pays off where it adds structure** (comparison, dropout), and gives diminishing returns where it only adds hand-crafted features.

The final method — self-attention over whole-trace embeddings with vote, arithmetic-error-count and final-answer-consistency features, regularised with dropout — reaches **92.98% on validation** at 1.26 MiB, and is one small CPU forward pass at inference.